Üretim Felsefesi: Deming'den Sola Kaydırmaya

ÖFY

October 2026

Özet

Kalite, güvenlik ve kullanıcı deneyimi, bitmiş bir ürüne sonradan eklenebilen katmanlar değil, ürünün üretilme biçiminin özellikleridir. Bir kusurun düzeltilme maliyeti, kusurun fark edildiği aşama ilerledikçe katlanarak artar; bu nedenle kusurları erken yakalayan süreçler, geç yakalayan denetimlerden ekonomik olarak üstündür. Son denetim bozuk ürünü ayıklar, ancak bozukluğu üreten nedeni ortadan kaldırmaz. Yazılımda bu ilke "sola kaydırma" (*shift left*) adıyla bilinir ve testin geliştirme sürecinin başına taşınmasını öngörür. Test güdümlü geliştirme (*Test-Driven Development*, TDD), bu ilkenin en uç biçimidir: test koddan önce yazılır ve kodun ne yapması gerektiğini tarif eden bir şartname işlevi görür. Yapay zekâ destekli geliştirmede kod üretimi ucuzlarken doğrulama pahalı kalır; bu durum testi geliştiricinin asıl katkısı hâline getirir. TDD testleri, bir sistemin durum uzayının tam bir gösterimi değil, o uzaydan alınmış seyrek bir örneklemdir; uzayı daha geniş kapsamak için özellik tabanlı test, model tabanlı test ve biçimsel doğrulama gibi yöntemler kullanılır.

Giriş

Geleneksel üretim anlayışında kalite, üretim tamamlandıktan sonra yapılan bir kontrol işlemi olarak görülür: ürün üretilir, ardından denetlenir, kusurlu olanlar ayrılır. Yazılımda bunun karşılığı, kodun aylarca yazılıp en sonda ayrı bir test ekibine teslim edilmesidir.

Bu notun karşı çıktığı kabul, kalitenin ürüne sonradan eklenebileceği varsayımıdır. Savunulan görüş şudur: kalite, güvenlik ve deneyim süreç içine yerleştirilmediğinde ya çok yüksek maliyetle elde edilir ya da hiç elde edilemez.

Notta kullanılan merkezî terimler şunlardır:

- **Süreç içi kalite**: Kalitenin üretimin her adımında, adımın kendisi doğru yapılarak sağlanması.
- **Sola kaydırma** (*shift left*): Test, güvenlik ve kalite faaliyetlerinin proje zaman çizelgesinde başa doğru çekilmesi. Zaman çizelgesi soldan sağa çizildiğinde fikir ve tasarım solda, teslim sağda yer aldığından bu ad verilmiştir.
- **Test güdümlü geliştirme** (*Test-Driven Development*, TDD): Bir işlevin testinin, işlevin kodundan önce yazıldığı geliştirme yöntemi.
- **Durum uzayı** (*state space*): Bir sistemin bulunabileceği tüm durumların ve bu durumlar arasındaki geçişlerin oluşturduğu küme.

Not, Japon sanayisinin kalite kültürünü biçimlendiren savaş sonrası Amerikan kaynaklı yaklaşımlardan başlayıp yazılım geliştirmeye uzanan bir çizgiyi izler. Kapsam, bu yaklaşımların mantığı ile sınırlıdır; belirli bir kalite standardının ya da test aracının ayrıntılı kullanımı kapsam dışındadır.

Kalitenin Süreçteki Yeri

Tarihsel Arka Plan: TWI ve Deming

Süreç içi kalite düşüncesinin yaygınlaşmasında iki Amerikan kaynağı belirleyici olmuştur.

Training Within Industry (TWI)

TWI, İkinci Dünya Savaşı sırasında ABD'de geliştirilen bir işçi ve ustabaşı eğitim programıdır. Deneyimli işçilerin cepheye gitmesiyle fabrikalara gelen deneyimsiz iş gücünü hızla yetiştirmek amacıyla tasarlanmıştır. Program üç modülden oluşur:

- **İş öğretme** (*Job Instruction*): Bir işin hazırlama, gösterme, denetleme ve takip etme adımlarıyla öğretilmesi. Temel ilkesi, öğrenemeyen işçinin değil, öğretemeyen öğreticinin başarısız sayılmasıdır.
- **İş yöntemleri** (*Job Methods*): Mevcut işin her adımının sorgulanması ve gereksiz adımların kaldırılması, birleştirilmesi, yeniden sıralanması ya da basitleştirilmesi.
- **İş ilişkileri** (*Job Relations*): Çalışanlarla ilgili sorunların bilgi toplama, değerlendirme, harekete geçme ve sonucu kontrol etme adımlarıyla adil biçimde çözülmesi.

Savaş sonrası işgal döneminde program Japonya'ya aktarılmış ve Toyota başta olmak üzere birçok şirket tarafından benimsenmiştir (Dinero, 2005). Sürekli küçük iyileştirme anlamına gelen *kaizen* anlayışının kökü büyük ölçüde İş Yöntemleri modülüne dayanır.

W. Edwards Deming

Deming, 1950'den itibaren Japon yöneticilere istatistiksel kalite kontrolü üzerine seminerler veren Amerikalı bir istatistikçidir. Temel görüşleri şunlardır (Deming, 1986):

- Kalite, ürün bittikten sonra yapılan denetimle değil, üretim sürecinin içinde sağlanır.
- Süreçteki değişkenlik istatistiksel yöntemlerle ölçülmeli ve izlenmelidir.
- İyileştirme, Planla–Uygula–Kontrol Et–Önlem Al (*Plan–Do–Check–Act*, PDCA) döngüsüyle sürekli yapılmalıdır.

Japonya'da kalite alanındaki en prestijli ödül bugün hâlâ Deming Ödülü adını taşır. "Japon kalitesi" çoğu zaman samuray geleneğine bağlansa da yöntemsel temeli büyük ölçüde bu savaş sonrası aktarıma dayanır.

Geç Bulunan Kusurun Maliyeti

Bir kusurun düzeltilme maliyeti, kusurun fark edildiği aşama ilerledikçe hızla artar. Yazılım mühendisliğinde bu gözlem ilk kez sistematik biçimde Boehm (1981) tarafından verilerle desteklenmiştir. Pratikte bu ilişki çoğu zaman "1-10-100 kuralı" olarak özetlenir: tasarımda 1 birime düzeltilebilen bir kusur, üretimde 10, müşteriye ulaştıktan sonra 100 birime mal olur. Oranlar sektöre göre değişir; sabit kalan, artışın doğrusal değil katlanarak gerçekleşmesidir.

Örnek olarak bir otomobilin fren tasarımındaki bir hata ele alınabilir. Hata çizim aşamasında fark edilirse yalnızca tasarım belgesi değişir. Fabrikada fark edilirse kalıplar yenilenir ve üretilmiş parçalar hurdaya ayrılır. Araçlar satıldıktan sonra fark edilirse yüz binlerce aracın geri çağrılması, olası kazalara bağlı hukuki süreçler ve marka itibarında kalıcı hasar gündeme gelir.

Denetim Kaliteyi Yaratmaz

Son denetim, kusurlu ürünü sağlam üründen ayırır; ancak kusuru üreten neden süreçte yerinde kaldığı için ertesi gün aynı kusur tekrar ortaya çıkar. Deming'in en bilinen önermelerinden biri budur: denetim kaliteyi artırmaz, yalnızca kalitesizliği kayıt altına alır (Deming, 1986).

Ekmeklerinin her gün yüzde yirmisini yanık olduğu için atan bir fırın bu durumun basit bir örneğidir. Kapıya bir kontrolcü yerleştirmek yanık ekmekleri ayırır, ancak yanmayı durdurmaz. Sorunu çözen, fırının sıcaklık ayarının düzeltilmesidir.

Sonradan Eklenemeyen Özellikler

Bazı özellikler, sistemin yapısal temeline bağlı olduğu için sonradan eklenemez. Bir binanın depreme dayanıklılığı temelinde ve taşıyıcı sisteminde belirlenir; bina tamamlandıktan sonra depreme dayanıklılık "eklenemez". Yazılımda da benzer biçimde, her kullanıcıya her veriye erişim veren bir mimari üzerine sonradan yetkilendirme katmanı eklemek, çoğu durumda sistemin büyük bölümünün yeniden yazılması anlamına gelir.

Ayrıca bitmiş üründe kusurlar görünmez hâle gelir. Kaynaklı bir parçadaki çatlak, parça boyandıktan sonra görülemez; bir güvenlik açığı binlerce satır kodun arasında kaybolur. Her adımda yapılan kontrol, kusuru daha küçük ve daha görünürken yakalar.

Sola Kaydırma ve Tasarımla Gelen Kalite

Sola Kaydırma İlkesi

"Sola kaydırma" (*shift-left testing*) terimi Larry Smith (2001) tarafından yazılım testinin geliştirme sürecinin başına taşınmasını tarif etmek için kullanılmıştır. İlke, testle sınırlı kalmayıp güvenlik, performans ve kullanıcı deneyimi gibi alanlara da genişlemiştir.

Farklı Alanlardaki Karşılıkları

Aynı düşünce farklı alanlarda "tasarımla" (*by design*) ekiyle anılan yaklaşımlar olarak karşımıza çıkar:

- **Tasarımla güvenlik** (*security by design*): Güvenlik gereksinimlerinin mimarinin ilk aşamasında belirlenmesi.
- **Tasarımla gizlilik** (*privacy by design*): Kişisel veri korumasının sistemin tasarımına baştan yerleştirilmesi. Kavram Cavoukian (2009) tarafından ilkeler hâlinde formüle edilmiş, Avrupa Birliği Genel Veri Koruma Tüzüğü'nün (GDPR) 25. maddesiyle hukuki yükümlülük hâline getirilmiştir (Avrupa Birliği, 2016).
- **Tasarımla kalite** (*quality by design*): Juran (1992) tarafından genel bir kalite yaklaşımı olarak tanımlanmış, ilaç sanayisinde ICH Q8 kılavuzuyla kurumsallaşmıştır. Kalite, ürün test edilerek değil, üretim sürecinin parametreleri baştan belirlenerek güvence altına alınır.
- **Üretilebilirlik için tasarım** (*design for manufacturing*): Ürünün kolay ve hatasız üretilebilecek biçimde tasarlanması.

Yazılımda Testin Dağılması

Testin sürecin sonundan başına taşınması, yazılımda birkaç uygulama ile gerçekleşir:

- **Test güdümlü geliştirme**: Kod yazılmadan önce testinin yazılması.
- **Sürekli entegrasyon** (*Continuous Integration*, CI): Her kod değişikliğinde testlerin otomatik olarak çalıştırılması. Kusur, yazıldığı gün yakalanır.
- **Kod incelemesi** (*code review*): Kodun ana dala birleştirilmeden önce başka bir geliştirici tarafından incelenmesi.

Kullanıcı deneyimi de bu ilkenin kapsamındadır. Kullanıcı odaklı tasarımda kullanıcılarla baştan görüşülür, kâğıt üzerinde taslaklar gösterilir ve erken sürümler denenir; deneyim, ürün bittikten sonra uygulanan bir görsel iyileştirme olarak ele alınmaz.

İlkenin Sınırı: Geri Bildirim Döngüsü

Her şey başlangıçta bilinemez. Kullanıcılar öngörülmeyen biçimlerde davranır, sistemler beklenmeyen yüklerle karşılaşır. Bu nedenle modern yaklaşım iki bileşenden oluşur:

- Bilinen gereksinimler baştan sürece yerleştirilir.
- Bilinemeyenler için geri bildirim döngüsü kurulur: izleme, kullanıcı geri bildirimi, küçük ve sık sürümler.

Bu yapı, Deming'in PDCA döngüsünün yazılımdaki karşılığıdır: süreç yalnızca baştan doğru yapmayı değil, sürekli öğrenip düzeltmeyi de içerir.

Test Güdümlü Geliştirme

Tanım ve Amaç

TDD, Kent Beck (2002) tarafından sistematik biçimde tanımlanmış bir geliştirme yöntemidir. İlk bakışta çelişkili görünür: henüz var olmayan bir kodun testi yazılır. Ancak yöntemin asıl işlevi test etmek değil, kodun ne yapması gerektiğini kod yazılmadan önce kesinleştirmektir.

Terzilik bu ilişki için açıklayıcı bir benzetme sunar. Terzi dikime başlamadan önce ölçü alır; ölçü alınmadan dikilen bir ceketin uyup uymayacağı tesadüfe kalır. TDD'de test ölçünün, kod ise ceketin karşılığıdır.

Kırmızı–Yeşil–Temizle Döngüsü

TDD üç adımlık bir döngüyle işler (Beck, 2002):

1. **Kırmızı**: Test yazılır. Kod henüz olmadığı için test başarısız olur.
2. **Yeşil**: Testi geçirecek en basit kod yazılır.
3. **Temizle** (*refactor*): Kodun yapısı iyileştirilir. Test hâlâ geçiyorsa davranış bozulmamıştır.

Örnek

Saniye cinsinden bir süreyi `dakika:saniye` biçimine çeviren bir işlev gerektiğinde, Go dilinde önce şu test yazılır:

```
func TestFormatDuration(t *testing.T) {
    cases := map[int]string{
        0:   "0:00",
        5:   "0:05",
        65:  "1:05",
        600: "10:00",
        -3:  "0:00", // negatif gelirse ne olsun?
    }
    for in, want := range cases {
        if got := FormatDuration(in); got != want {
            t.Errorf("FormatDuration(%d) = %s, want %s", in, got, want)
        }
    }
}
```

Testin yazılması, kod yazılırken çoğu zaman atlanacak kararların açıkça verilmesini zorunlu kılar: beş saniyenin `0:5` mi yoksa `0:05` mi gösterileceği, negatif bir değer geldiğinde ne yapılacağı. Kusurların büyük bölümü bu tür köşe durumlarında (*edge cases*) ortaya çıkar.

Yapay Zekâ Destekli Geliştirmede TDD

Yapay zekâ modelleri kodu çok hızlı üretir, ancak sözdizimsel olarak doğru, derlenen ve düzgün görünen bir kod belirli bir köşe durumunda yanlış davranabilir. Bu durumda testin üç işlevi öne çıkar:

- **Sözleşme işlevi**: Test, modele verilen davranış şartnamesidir. Üretilen kodun doğruluğu yoruma değil, testin geçip geçmediğine bağlanır.
- **Gerileme** (*regression*) **tespiti**: Model bir bölümü düzeltirken başka bir bölümü bozabilir. Mevcut testler bu bozulmayı anında görünür kılar.
- **İnceleme yükünün azalması**: Modelin ürettiği yüzlerce satırı satır satır incelemek yerine, kısa ve okunabilir testlerin doğru davranışı tarif edip etmediği denetlenir.

Bu koşullarda geliştiricinin rolü kod yazmaktan, neyin doğru olduğunu tarif etmeye kayar; test bu tarifin en kesin biçimidir.

Sınırlılıklar

TDD her bağlamda gerekli değildir. Tek seferlik betikler, hızlı denemeler ve ne yapılacağının henüz bilinmediği keşif amaçlı kodlar için önceden test yazmak maliyetini karşılamayabilir. Yöntemin getirisi, mantığı kalıcı olacak, başkalarınca kullanılacak ve sık değişecek kodda belirgin biçimde yüksektir.

TDD ve Durum Uzayı

Benzerlik

Bir işlevin alabileceği tüm girdiler ve üretebileceği tüm çıktılar bir uzay olarak düşünülebilir. TDD testleri bu uzayda belirli noktaları sabitler: "bu girdi geldiğinde bu çıktı üretilmelidir". Durumlu bir sistemde testler "şu durumdayken şu olay gelirse sistem şu duruma geçmelidir" biçimini alır; bu, durum geçişlerinin tarif edilmesine benzer.

Bir diğer bakış açısıyla her test, kabul edilebilir programlar uzayını daraltır. Hiç test yokken her program kabul edilebilirdir; eklenen her test, yanlış davranan programların bir kısmını eler. Bu açıdan TDD, çözüm uzayının adım adım kısıtlanması olarak görülebilir.

Ayrım

Kontrol teorisindeki durum uzayı gösterimi ya da bir sonlu durum makinesi (*finite state machine*), sistemin tüm davranışını tarif eder: her durum ve her geçiş tanımlıdır. TDD testleri ise yalnızca örneklerdir. Beş nokta sabitlendiğinde, bu noktaların arasında kalan girdilerde sistemin nasıl davranacağı hakkında test hiçbir şey söylemez.

Bu sınır Dijkstra (1970) tarafından genel biçimde ifade edilmiştir: test, kusurların varlığını gösterebilir, ancak yokluğunu hiçbir zaman kanıtlayamaz. Bir benzetmeyle durum uzayı gösterimi bir şehrin tam haritasına, TDD testleri ise bu haritadaki birkaç kontrol noktasına karşılık gelir.

Uzayı Daha Geniş Kapsayan Yöntemler

Durum uzayını daha kapsamlı biçimde ele almak için şu yöntemler kullanılır:

- **Özellik tabanlı test** (*property-based testing*): Tek tek örnekler yerine genel kurallar yazılır; örneğin "her pozitif girdi için çıktı belirli bir biçimde olmalıdır". Araç binlerce rastgele girdi üreterek kuralı dener. Yaklaşım Haskell için geliştirilen QuickCheck aracıyla yaygınlaşmıştır (Claessen ve Hughes, 2000). Go dilinde standart kütüphanedeki `testing/quick` paketi ve `rapid` kütüphanesi bu amaçla kullanılır.
- **Model tabanlı test** (*model-based testing*): Sistemin durum makinesi açıkça modellenir ve test senaryoları bu modelden otomatik olarak üretilir. Bu, durum uzayı gösteriminin doğrudan test amacıyla kullanılmasıdır.
- **Biçimsel doğrulama** (*formal verification*): TLA+ gibi araçlarla sistemin tüm durum uzayı matematiksel olarak taranır (Lamport, 2002). En kapsamlı ancak en zahmetli yöntemdir.

Sonuç

Kalitenin süreç içine yerleştirilmesi bir tercih değil, maliyet yapısının dayattığı bir zorunluluktur. Kusurun düzeltilme maliyeti geç aşamalarda katlanarak arttığı ve bazı özellikler yapısal temele bağlı olduğu için, son denetime dayanan bir kalite anlayışı ya pahalıdır ya da yetersizdir. Bu gerekçe imalattan yazılıma, güvenlikten gizliliğe kadar farklı alanlarda aynı biçimde işler; "tasarımla" ekiyle anılan yaklaşımların çeşitliliği, bunun tek bir sektöre özgü bir gözlem olmadığını gösterir.

TDD'nin değeri, test etmekten çok şartname yazmaya zorlamasındadır. Bu nitelik, yapay zekâ destekli geliştirmede belirleyici hâle gelir: kod üretimi ucuzladıkça darboğaz doğrulamaya kayar ve doğruluğun tarifi geliştiricinin asıl katkısı olur.

Çerçevenin iki sınırı vardır. Birincisi, her şey baştan bilinemez; süreç içi kalite, gerçek kullanımdan öğrenen bir geri bildirim döngüsüyle tamamlanmadıkça eksik kalır. İkincisi, TDD testleri durum uzayının seyrek bir örneklemidir ve sabitlenen noktaların dışında güvence vermez. Örnek tabanlı testin nerede yeterli olduğu, özellik tabanlı ya da biçimsel yöntemlere hangi noktada geçilmesi gerektiği, sistemin kritiklik düzeyine bağlı açık bir sorudur. Yapay zekâ modellerinin testleri de üretebildiği durumda, testin bağımsız bir doğrulama aracı olma niteliğinin nasıl korunacağı ise henüz yerleşik bir cevabı olmayan bir sorundur.

Kaynaklar

- Avrupa Birliği (2016). Regulation (EU) 2016/679 (General Data Protection Regulation), Madde 25: Data protection by design and by default.
- Beck, K. (2002). *Test-Driven Development: By Example*. Addison-Wesley.
- Boehm, B. W. (1981). *Software Engineering Economics*. Prentice Hall.
- Cavoukian, A. (2009). *Privacy by Design: The 7 Foundational Principles*. Information and Privacy Commissioner of Ontario.
- Claessen, K. ve Hughes, J. (2000). QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. *Proceedings of ICFP 2000*.
- Deming, W. E. (1986). *Out of the Crisis*. MIT Press.
- Dijkstra, E. W. (1970). *Notes on Structured Programming* (EWD249).
- Dinero, D. A. (2005). *Training Within Industry: The Foundation of Lean*. Productivity Press.
- ICH (2009). *Q8(R2): Pharmaceutical Development*. International Council for Harmonisation.
- Juran, J. M. (1992). *Juran on Quality by Design*. Free Press.

- Lamport, L. (2002). *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley.
- Smith, L. (2001). Shift-Left Testing. *Dr. Dobb's Journal*.